



沈阳建筑大学

上机实验报告

课程名称:

操作系统

实验题目:

银行家算法模拟实验

专业班级:

计算机 2101 班

学 号:

2104230414

日 期:

2024.04.17

【实验目的】

1. 理解死锁的概念；
2. 用高级语言编写和调试一个银行家算法程序，以加深对死锁的理解。

【实验准备】

1. 产生死锁的原因
 - 竞争资源引起的死锁
 - 进程推进顺序不当引起死锁
2. 产生死锁的必要条件
 - 互斥条件
 - 请求和保持条件
 - 不剥夺条件
 - 环路等待条件
3. 处理死锁的基本方法
 - 预防死锁
 - 避免死锁
 - 检测死锁
 - 解除死锁

【实验内容】

1. 实验原理

银行家算法是从当前状态出发，逐个按安全序列检查各客户中谁能完成其工作，然后假定其完成工作且归还全部贷款，再进而检查下一个能完成工作的客户。如果所有客户都能完成工作，则找到一个安全序列，银行家才是安全的。与预防死锁的几种方法相比较，限制条件少，资源利用程度提高了。缺点：该算法要求客户数保持固定不变，这在多道程序系统中是难以做到的；该算法保证所有客户在有限的时间内得到满足，但实时客户要求快速响应，所以要考虑这个因素；由于要寻找一个安全序列，实际上增加了系统的开销。Banker algorithm最重要的一点是：保证操作系统的安全状态！这也是操作系统判断是否分配给一个进程资源的标准！那什么是安全状态？举个小例子，进

程 P 需要申请 8 个资源（假设都是一样的），已经申请了 5 个资源，还差 3 个资源。若这个时候操作系统还剩下 2 个资源。很显然，这个时候操作系统无论如何都不能再分配资源给进程 P 了，因为即使全部给了他也不够，还很可能造成死锁。若这个时候操作系统还有 3 个资源，无论 P 这一次申请几个资源，操作系统都可以满足他，因为操作系统可以保证 P 不死锁，只要他不把剩余的资源分配给别人，进程 P 就一定能顺利完成任务。

2. 实验题目

设计五个进程{P0, P1, P2, P3, P4}共享三类资源{A, B, C}的系统，{A, B, C}的资源数量分别为 10, 5, 7。进程可动态地申请资源和释放资源，系统按各进程的申请动态地分配资源。要求程序具有显示和打印各进程的某一时刻的资源分配表和安全序列；显示和打印各进程依次要求申请的资源号以及为某进程分配资源后的有关资源数据。

3. 算法描述

我们引入了两个向量：**Resource**（资源总量）、**Available**（剩余资源量）以及两个矩阵：**Claim**（每个进程的最大需求量）、**Allocation**（已为每个进程分配的数量）。它们共同构成了任一时刻系统对资源的分配状态。

向量模型：

R1	R2	R3
----	----	----

矩阵模型：

	R1	R2
P1		
P2		
P3		

这里，我们设置另外一个矩阵：各个进程尚需资源量（Need），可以看出

$$\text{Need} = \text{Claim} - \text{Allocation}$$

因此，我们可以这样描述银行家算法：

设 $Request[i]$ 是进程 P_i 的请求向量。如果 $Request[i, j]=k$, 表示 P_i 需 k 个 R_j 类资源。当 P_i 发出资源请求后, 系统按下述步骤进行检查:

(1) if ($Request[i] \leq Need[i]$) goto (2);

 else error("over request");

(2) if ($Request[i] \leq Available[i]$) goto (3);

 else wait();

(3) 系统试探性把要求资源分给 P_i (类似回溯算法)。并根据分配修改下面数据结构中的值。

$Available[i] = Available[i] - Request[i]$;

$Allocation[i] = Allocation[i] + Request[i]$;

$Need[i] = Need[i] - Request[i]$;

(4) 系统执行安全性检查, 检查此次资源分配后, 系统是否处于安全状态。若安全, 才正式将资源分配给进程以完成此次分配; 若不安全, 试探方案作废, 恢复原资源分配表, 让进程 P_i 等待。

系统所执行的安全性检查算法可描述如下:

设置两个向量: $Free$ 、 $Finish$

工作向量 $Free$ 是一个横向量, 表示系统可提供给进程继续运行所需要的各类资源数目, 它含有的元素个数等于资源数。执行安全算法开始时, $Free = Available$. 标记向量 $Finish$ 是一个纵向量, 表示进程在此次检查中是否被满足, 使之运行完成, 开始时对当前未满足的进程做 $Finish[i] = false$; 当有足够资源分配给进程($Need[i] \leq Free$)时, $Finish[i] = true$, P_i 完成, 并释放资源。

(1) 从进程集中找一个能满足下述条件的进程 P_i

① $Finish[i] == false$ (未定)

② $Need[i] \leq Free$ (资源够分)

(2) 当 P_i 获得资源后, 认为它完成, 回收资源:

$Free = Free + Allocation[i]$;

$Finish[i] = true$;

Go to step(1);

试探此番若可以达到 $Finish[0..n] := true$, 则表示系统处于安全状态, 然后再具体为申请资源的进程分配资源。否则系统处于不安全状态。

我们还举银行家的例子来说明：设有客户 A、B、C、D，单一资源即为资金（R）。

下列状态为安全状态，一个安全序列为：C->D->B->A

A	1	6
B	1	5
C	2	4
D	4	7

Available = (2) ; Resourse = (10) ;

【源程序代码】

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <iostream>

using namespace std;

#define N 10
#define M 10

typedef struct{
    int ip;
    int able[N];
    int visited[N];
    int remain[M];
}WorkNode;    /*工作节点*/

/*ass 为进程已分配资源,need 为最大需求,r 为各种资源的个数*/
int ass[N][M],need[N][M],r[M],top=-1,pnum=0,rnum=0;

/*currnet 为当前工作节点*/
WorkNode stack[N],current;

int main()
{
```

```

int i,ct = 0;

void add(int [],int []);      /*进程完成,释放资源*/
int small(int [],int []);    /*已有的资源是否能满足最大请求*/
void init();                  /*初始化*/
void output();
//void output();             /*输出堆栈中的一种可能*/
int bankalgo();               /*银行家算法主要部分，回溯法*/

init();
for(i=0;i<pnum;i++)
    if(small(need[i],current.remain) )
        current.able[++current.ip]=i;
if(current.ip== -1)
{
    printf("it is unsafe\n");
    return 0;
}
ct = bankalgo();
printf("\nit is safe,and it has %d solutions\n",ct);
//else printf("\nit is unsafe\n");

return 0;
}

void init()
{
    int i,j,sum=0;
    system("cls");
    //clrscr();
    printf("process number:");
    scanf("%d",&pnum);
    printf("resource number:");
    scanf("%d",&rnum);
    printf("resource series:");
    for(i=0;i<rnum;i++)
        scanf("%d",&r[i]);
    printf("assined matrix:");
    for(i=0;i<pnum;i++)

```

```

{
    printf("p%d:",i);
    for(j=0;j<rnum;j++)
        scanf("%d",&ass[i][j]);
}
printf("needed matrix:\n");
for(i=0;i<pnum;i++)
{
    printf("p%d:",i);
    for(j=0;j<rnum;j++)
        scanf("%d",&need[i][j]);
}
memset(current.visited,0,sizeof(current.visited));
for(i=0;i<rnum;i++)
{
    for(j=0;j<pnum;j++)
        sum+=ass[j][i];
    current.remain[i]=r[i]-sum;
    sum=0;
}
current.ip=-1;
}

void output()
{
    int i;
    for(i=0;i<=top;i++)
        printf("p%d,", stack[i].able[stack[i].ip]);
    printf("\n");
}

void add(int x[],int y[])
{
    int i;
    for(i=0;i<rnum;i++)
        x[i]+=y[i];
}

int small(int x[],int y[])

```

```

{
    int i;
    for(i=0;i<rnum;i++)
    {
        if(x[i]>y[i])break;
    }
    if(i==rnum)return 1;
    return 0;
}

int bankalgo()
{
    int i,ct=0;
    while(1)
    {
        stack[++top]=current;
        current.visited[current.able[current.ip]]=1;
        add(current.remain,ass[current.able[current.ip]]);
        current.ip=-1;
        for(i=0;i<pnum;i++)
            if(!current.visited[i] && small(need[i],current.remain) )
                current.able[++current.ip]=i;
        if(current.ip==0)
        {
            if(top==pnum-1)
            {
                output();
                ct++;
            }
            else if(top<0) break;
            current=stack[top--];
            while(current.ip==0) current=stack[top--];
            current.ip--;
        }

        if (top < 0) break;
    }
    return ct;
}

```


运行结果截图：

```
process number:5
resource number:4
resource series:6 3 4 2
assigned matrix:p0:3 0 1 1
p1:0 1 0 0
p2:1 1 1 0
p3:1 1 0 1
p4:0 0 0 0
needed matrix:
p0:1 1 0 0
p1:0 1 1 2
p2:3 1 0 0
p3:0 0 1 0
p4:2 1 1 0
p3 -> p4 -> p0 -> p2 -> p1 ->
p3 -> p4 -> p0 -> p1 -> p2 ->
p3 -> p0 -> p4 -> p2 -> p1 ->
p3 -> p0 -> p4 -> p1 -> p2 ->
p3 -> p0 -> p2 -> p4 -> p1 ->
p3 -> p0 -> p2 -> p1 -> p4 ->
p3 -> p0 -> p1 -> p4 -> p2 ->
p3 -> p0 -> p1 -> p2 -> p4 ->

it is safe,and it has 8 solutions

-----
Process exited after 49.62 seconds with return value 0
请按任意键继续. . .
```

【实验体会】

这是第二次操作系统实验，我学习了银行家算法，是我对死锁的预防有了更加深刻的理解。对于程序的现存状态，先去判断是否处于合理状态，检测到合理之后，再使用银行家算法试着去分配，检查是否处于安全序列（能否找到一个序列把所有资源依次分配，直到全部分配），如果处于安全序列就证明这次分配是合理的，找不到安全序列就证明是不合理的，就回收之前分配的，让这次请求进入等待序列。

银行家算法是操作系统的一个重要算法，学会这个对我得知识水平和算法实践能力都有不小的提升。

【思考题】

银行家算法有什么缺点？

1. **适用性有限：**银行家算法适用于资源预分配的系统，即系统能够预先知道每个进程的最大资源需求。但在许多实际情况下，进程的最大资源需求是动态变化的，这使得银行家算法的应用受到限制。
2. **资源利用率可能不高：**由于银行家算法较为保守，它倾向于保留资源以确保系统安全，这可能导致资源的利用率不是最高的。在实际应用中，可能会出现系统中有大量资源被保留而无法有效利用的情况。
3. **饥饿问题：**在银行家算法中，如果一个进程请求的资源暂时无法满足，它必须等待。如果这种等待无限期地持续下去，就可能导致进程饥饿，即进程长时间得不到所需资源而无法继续执行。

4. **响应时间可能较长：**由于银行家算法需要确保系统安全，当一个进程请求资源时，系统需要检查这次分配是否会导致系统进入不安全状态。这个过程可能导致进程的响应时间变长。
5. **实现复杂：**银行家算法的实现相对复杂，需要精确跟踪系统中每个进程的状态以及可用资源，这增加了系统的复杂性。